IT-PEP

ZGP

Development Specification (Business and Technical)

Naming Conventions

INTERNAL
INTERN

Person Responsible:	Martina Muster, K-SIx-y
Status:	approved
Version:	V1.0
Date:	30.11.2022
Template-Version:	2.1.1
IT-PEP Version:	2.2.2
Person Responsible:	Martina Muster, K-SIx-y

Classification System for Data

(Klassifizierungssystematik für Unterlagen – KSU):

Class	Class Shortcut	Retention Period

Version History:

Version	Date	Author	Comments
V1.0	25.04.2024	Rohit Kadam	Initial version

Distribution:

Name	Company/Area/Department

Table of contents

1. Scope

This regulation applies to all employees who work in the IT department of one of the companies of Volkswagen Financial Services AG and who develop in the SAP environment.

2. Objective

The development guideline for SAP sets binding standards to:

- Support the developers with clear guidelines
- Simplify the care and maintenance of applications
- Improve the quality of applications

Validity
The development guideline applies to all applications developed and operated on SAP platforms by Volkswagen Business Services GmbH.

Responsibility
The guideline is binding, compliance must be ensured by the respective developer.

Exceptions
Exceptions must be requested and approved by the SAP Committee.

Changes
Changes to the policy are made by the SAP Committee.
Requests for amendments must be submitted via the SAP Committee Chair.

3. Naming conventions for ABAP and ABAP-OO

All new repository objects (DDIC and program objects) must be created in the /VWK/ namespace (as far as possible). The general structure of the identifiers is: /VWK/&&&* (&&& = application).SAP

Currently, there are (examples):

Development	FS Abbreviation	Description	Contact
CIC	/VWK/CRM/VWK/CFS	CRM	I-SEC
ZGP	/VWK/ZGP	Central Business Partner	I-SEZ
XI	/VWK/SKS	System Communication Interface	I-SEG
SFA	/VWK/SFA	Service Factory	I-SEG
ZFW	/VWK/ZFW	Transaction Framework	I-SEG

An exception is made for applications or developments that were previously created in the Z namespace (e.g. ZF* for SAP FI). New objects for these applications are created in the previous namespace. The requirements of chapter 3.3 and 3.5 are to be adapted accordingly. Chapter 3.6 is universally valid.
The use of namespaces of other applications/developments is not allowed.

- For transportable objects use **/VWK/namespace**:
  - / VWK / nnn…; nnn = separation abbreviation
  - Currently usable separation abbreviations:
    - ZGP: Central business partner
    - AKB: Information processing
    - BON: Creditworthiness
    - ZDC: ZGP data cleansing
    - ZAR Central address research
    - ZVT zContract

- **local objects in the /VWK/namespace** should not be created.
- Clarify the classification of new objects with the person responsible for the parcel

3.1 Objects to be documented

If SAP provides object-specific documentation via SAPScript (Call via button "Documentation" or in the menu bar “Jump" → "Documentation"), these objects have to be documented according to the a.o. guideline. Cross-references to other documentation objects should be provided via
After a collective transport has been imported into the F3P, all SAPScript documents (applications and objects) are made available with a SAP report as HTML documents in the GoBS folder.
Links in the SAPScript.

Reports, Form routines and PBO / PAI modules are by inserting appropriate comment lines (see. Documentation in the coding) document in your head.

Object	**Document type**	**SAPScript docu**	**Brief description in the documentation header**	**Detailed description in the documentation Header**
Database table	Technical	**Required**		
View	Technical	**Required**		
Structure	Technical	**Required**		
Data element	User	**Required**		
Function block	Technical	**Required**	**Required**	
Function block parameters	Technical	optional		
Report	User / Technical	**Required / Required (see below)**	**Required**	**Required**
Form routine	Technical		**Required**	**Required**
PAI / PBO module	Technical		**Required**	**Required**
Class	Technical	**Required**		
Method	Technical	**Required**	**Required**	
Class attributes	Technical	optional		
Class events	Technical	**Required**		
Business Add Ins (Def.)	Technical	**Required**		
Business Add-Ins (impl.)	technical	**Required**		

The **SAPScript documentation of reports** is actually - as far as the given headings are concerned - an **operation manual** for execution. In order to display the general **technical description** in

HTML for comprehension and maintenance the report, if it is not stored in detail at the subroutine level, it is attached as shown at the end of the SAPScript docu:

- after the chapter &EXAMPLE& insert a new line in paragraph format **Heading 1**
- then insert the symbol **&Z_ZGP_TECH_DOKU&** (heading "Technical Documentation") by clicking on the "Insert command" button
- then document in a structured manner using the remaining headings, bullets and character formats

3.2 How is it to be documented?

3.2.1 Program objects

All Program objects (Report, Function module, form routine, method, Module) have especially functional and technical background ("**Why was the functionality implemented this way?“**) in the foreground, less the "What? ". This has to be described sparse over Documentation in the coding - unless it is obvious.

When describing the program objects care must be taken that, the **essential information on processing is documented** on the same Hierarchy level:

- In the function module, if the main processing is carried out there
- In the form routine, if it is called several times

Technical features that are not necessarily apparent from the coding have to be described ("**How?**").

Deviations from the above-mentioned Development guideline (e.g. **Use of global variables**) must be justified here.

For reasons of transparency, also the **Use of ABAP or SAP memories** have to be described.

The documentation always has to use always the given headlines. Their deletion is not allowed. Sections that are not used can be left blank. **The description must be understandable and in full sentences.**

3.2.2 Interfaces of program objects

Interfaces of Program objects are to be documented using the available options in accordance with the following list.

Object	Short text / Description	Long text
Function block	From DDIC object; adjust manually if necessary	Using the given headlines
Form routine	Manual adaptation of the templates when creating a new system with forward navigation	
Method	Description of Data element designation; adjust manually if necessary; For a detailed description, use the "Parameters" section of the method documentation	

3.2.3 Applications

Applications are documented in the SAP script using transaction SE61. This is necessary because complex **relationships, processes and components** involved within an application in SAP cannot be properly represented at the object level.

A sample (CHAP/VWK/ ZGP_MUSTER_ANW) is available to help create a new SAP Script document.

3.2.4 Provision of the SAPScript documents in the GoBS folder

After a collective transport has been imported into the F3P, all SAPScript documents (applications and objects) are made available with a SAP report as HTML documents in the GoBS folder.

3.2.5 Links in the SAPScript

In order to insert links in the SAPScript, you can either create a link in the SAPScript editor via the menu Include-> Reference (but then you may not have all link types available or you can enter the link directly in the **old** SAPScript editor.
With the route via Include-> Reference, you can only include existing documentation, with the manual route, the linked documentation does not have to exist yet (you only get a note when activating the documentation, which can be ignored).

When entering manually, a link is given as follows:
<DS:*linktyp*.*objekt*>*linktext*</>

Object is the name of the linked object (e.g. the name of a data element).
The link text is the text that is then displayed in the documentation text.

Link type	**Documentation**
TB	Table, structure, view
DE	Data element
DO	Domain
CL	Class
CO	Class method - The class name must be specified here as the object (padded with spaces up to length 30), followed by the method name E.g.: <DS:CO./VWK/CL_ZGP_MA_PARTNER SAVE>SAVE</>
CA	Class attribute - The class name must be specified here as the object (padded with spaces up to length 30), followed by the attribute name
CE	Class event - The class name must be specified here as the object (padded with spaces up to length 30), followed by the event name
IF	Interface
IA	Interface attribute - The interface name must be specified here as the object (padded with spaces up to length 30), followed by the attribute name
IE	Interface event - The interface name must be specified here as the object (padded with spaces up to length 30), followed by the event name
IO	Interface method - The interface name must be specified here as the object (padded with spaces up to length 30), followed by the method name
FU	Function block
FU	Function block parameters - The function name must be specified here as the object (padded with spaces up to length 30), then the parameter name E.g.: <DS:FU./VWK/ZGP_MA_SCR_SET_GP IV_PARTNER>IV_PARTNER</>

FX	Function block exceptions - The function name must be specified here as the object (padded with spaces up to length 30), followed by the exception name
N / A	Message - Here the object name consists of the message class followed by the number (without spaces)
NC	Message class
RE	Report / module pool / include / function module group (specify the name of the framework program for function groups, i.e. for FG / VWK / ZGP_MY_FUNCS -> / VWK / SAPLZGP_MY_FUNCS)
EQ	Lock object
TX	General text

Example:
<DS:TB./VWK/ZGPD_MAAUT>table</>

A special link is the one to execute a transaction:

Link type	**Documentation**
TRAN	Executing a transaction
TRAS	Execute a transaction and skip first screen

Example in a table documentation
&MAINTENANCE&
<DS.TRAN./VWK/ZGP_SM30TABLE>run</>

3.3 Documentation in the coding

3.3.1 Inline documentation

- Where it is necessary for a direct understanding of the coding
- Also here just sparse "what?", more like "why?"
- As a theme **"As little as possible - as much as necessary!"**
- Create detailed program descriptions in GOBS (especially detailed "**Why?**")
- Complex relationships (if necessary display graphically) under Processes

3.3.2 Documentation header - "Header"

For the following development objects, a documentation header must be inserted and maintained at the top

- Report
- Function block
- Form routine / subroutine
- Method

Here, the developer should be able to see the most important information at first glance

- Surname
- Basic functions description
- Creator and creation date
- Reason for the development (project, incident, WOM)

- Change history

NOTE:
The change history is NOT the right place for detailed documentation. It should only describe the most important changes in 1-2 lines.
If basic functions change, this must be adapted in the function description and the SAP-Script-documentation!

It is recommend using the available Templates via the SAP pattern function in the editor, accessible via "Pattern" -> "Insert pattern" -> "Other pattern" because the current user and date are automatically inserted here:

- New installation complete: ZZGP_HEAD_00
- New installation Creation info: ZZGP_HEAD_01
- Modification information: ZZGP_HEAD_02

If necessary, adjustments and the creation of additional samples can also be made in the associated function blocks [NAME] _EDITOR_EXIT in the local function group Z_ZGP_DEVELOPMENT.

3.3.2.1 Header example - Report

```
*&---------------------------------------------------------------------*
*
*&      Report  ZABE_TEST_LOG_OFF
*&---------------------------------------------------------------------*
*
*  Testprogramm für FuBas zum Abmelden bzw. Task-Handler
*  (z.B. Modus schließen)
*---------------------------------------------------------------------*
*
* 001 2008-05-30 [IM000815] Alfred Behrends (DKX0NNC)
*     Erstellt
* 002 2008-12-02 [W004711] Alfred Behrends (DKX0NNC)
*     Anpassung asynchroner Aufruf
*---------------------------------------------------------------------*
*
program y_multiple_programs_check.
…
```

3.3.2.2 Header example - function block

```
*&---------------------------------------------------------------------*
*
*&      Function  ZZDC_HEAD_00_EDITOR_EXIT
*&---------------------------------------------------------------------*
*
*  ZDC/DCM Editor-Exit für die Musterfunktion zur Erstellung des
*  vollständigen Programmheaders bei Neuanlage/Erstellung
*---------------------------------------------------------------------*
*
* 001 2009-05-05 [PP47/11] Alfred Behrends (DKX0NNC)
*     Erstellt
* 002 2008-12-02 [W004711] Alfred Behrends (DKX0NNC)
```

```
*     Anpassung asynchroner Aufruf
*---------------------------------------------------------------------
*
function zzdc_head_00_editor_exit .
*"---------------------------------------------------------------------
-
*"*"Lokale Schnittstelle:
…
```

3.3.2.3 Header example - Subroutine

```
*&---------------------------------------------------------------------
*
*&      Form  HEAD_ULINE_GET
*&---------------------------------------------------------------------
*
*  Trennzeile für Kommentarzeile erstellen
*---------------------------------------------------------------------
*
* 001 2007-12-18 [W004711] Alfred Behrends (DKX0NNC)
*     Erstellt
* 002 2008-03-03 [ZGP] Alfred Behrends (DKX0NNC)
*     Anpassung Zeilenlänge
*---------------------------------------------------------------------
*
*     <--CV_COMMENTLINE  Kommentarzeile
*---------------------------------------------------------------------
*
form head_uline_get changing cv_commentline type text255.
```

3.3.2.4 Example method

```
*&---------------------------------------------------------------------
*
*&      Method VAB_CENTRAL_DATA
*&---------------------------------------------------------------------
*
*  DCA - Verarbeitung der Zentralen Daten
*  Daten aus den Tabellen BUT000 + BUT001
*---------------------------------------------------------------------
*
* 001 2009-05-05 [DC] Alfred Behrends (DKX0NNC)
*     Erstellt
*---------------------------------------------------------------------
*
method …
```

3.4 Naming Conventions for DDIC Objects

	Name	Description
DB Table	/VWK/&&&D_c... c/VWK/&&&C_ c... c	Application TableCustomizing Table

View	/VWK/&&&V_ c... c	
Table Type	/VWK/&&&T_ c... c	
Structure	/VWK/&&&S_ c... c	
Data item	/VWK/&&&_ c... c	
Domain	/VWK/&&&_ c... c	
Search Help	/VWK/&&&_ c... c	
Lock	/VWK/&&&_ c... c	
Type Group		Do not use

Legend:

&&&	Separation abbreviation
Cc... Cc	Arbitrary alphanumeric string
ccccc	Any alphanumeric string, up to a maximum of 5 characters

3.5 Naming Conventions for Other Repository Objects

With the following exceptions, the naming convention applies to all other repository objects (programs, includes, transactions, function groups,...): /VWK/&&&_ c... c
For objects that cannot be created in the /VWK/ namespace (e.g. BSP applications) the following applies: Z&&&_ c... c

	Name	**Description**
Class	/VWK/CL_&&&_c... c	
Interface	/VWK/IF_&&&_ c... c	
Exception	/VWK/CX_&&&_c... c	

Legend:

&&&	Separation abbreviation
Cc... Cc	Arbitrary alphanumeric string
ccccc	Any alphanumeric string, up to a maximum of 5 characters

Note: These exceptions are necessary because SAP prescribes the described naming for persistent classes and exception classes, among other things.

3.6 Naming Conventions Program Objects

Analogous to the naming conventions for DDIC objects, there are also conventions for naming objects within the program.

	Name	**Description**
Select-Options	S_ccccc	Selection screen report (max. 8 characters)

Parameters	P_cccccc	Selection screen report (max. 8 characters)
Types	T_cc... Cc TT_cc... Ccc	The type (felder, structure) Types (Tables)
Data	G$_cc... Cc	Global Data (Supporting Programme)
	O$§_cc... Cc	Instance Data (Object)
	S$§_cc... cc	Static Data (Class)
	L$_cc... Cc	Local data (subroutine, function module, method)
	S$_ cc... Cc	Static Data (Subroutine, Function Module)
Shape parameter	U$_cc... Cc	Using-Variable
	C$_cc... Cc	Changing-Variable
	#T_cc... Cc	Table handover
Fkt.baustein	I$_cc... Cc	Importing-Variable
	E$_cc... Cc	Exporting-Variable
	C$_cc... Cc	Changing-Variable
	#T_cc... Cc	Table handover
Memory	M$_cc... Cc	Memory-Variable (im ABAP)
	/VWK/&&&_cc... Cc	Memory-Variable (im Memory)
Classes	LCL_&&&_cc... Cc	Local Class
	LIF_&&&_cc... Cc	Local Interface
	I$_cc... Cc	Importing-Variable
	E$_cc... Cc	Exporting-Variable
	C$_cc... cc	Changing-Variable
	R$_cc... Cc	Returning Handover

Legend:

$ Data format C Konstante
V Variable (Single Field)R Reference to ObjectS StructureT Table
§ Visibility (objects only)U pUblicO prOtectedI Private
# Tables in interfaces (Form, Function), use of importing, exporting, changing variables with table types is preferable
I Importing
E Exporting
C Changing
&&& Separation abbreviation
Cc... Cc Arbitrary alphanumeric string
ccccc Any alphanumeric string, up to a maximum of 5 characters

General:

A variable/structure should always have the name of the field/table in the name to which it refers. A subroutine should always be meaningfully described by its function and object (analogous to the naming of methods).

3.7 Naming Conventions File Names

SAP file names are to be formed as follows:

- Input and output files:
  - Generate filename dynamically via transaction FILE (compare Programming tips)
  - Generate OK file with the same name (not for test run!):
    - For automatic further processing:
      - Transfer of the output file for further steps by UC4
      - Copy the Input- or the Output-, as well as the OK-file in the backup directory with subsequent deletion from the work directory by Unix-Script (ATTENTION! Script has to be adapted to new file names by I-SBB!)
    - Example for CCF-Output file that is further processed by Connect:Direct (COD):
      - Original file: **DE1030000CCFDELETCOD_OUT.F3P**
      - OK file: **DE1030000CCFDELETCOD_OUT.F3P.OK**
- Direclty using Open and Close dataset will lead into system dump, we need to always use **FILE_GET_NAME** and **FILE_NAME_CHECK** Functions before Dataset call.

```
FORM open_file CHANGING cv_file_name TYPE filename-fileintern.

  CALL FUNCTION 'FILE_GET_NAME'
    EXPORTING
      client           = sy-mandt
      logical_filename = p_filnm
      operating_system = sy-opsys
      parameter_1      = p_land
      parameter_2      = p_funk
      parameter_3      = p_herk
      eleminate_blanks = abap_true
    IMPORTING
      file_name        = cv_file_name
    EXCEPTIONS
      file_not_found   = 1
      OTHERS           = 2.

  IF sy-subrc <> 0.
    " Abbruch - Der Dateiname & ist unbekannt
    MESSAGE e001(sg) WITH cv_file_name.
  ENDIF.
```

3.8 Naming Conventions Creator Names for Batch Input Sessions

To distinguish whether batch input sessions of jobs have been created, the name of a technical user should be set in the creator name:

Valid technical users are:

- BATCHADMIN
- CONTROL_M
- CONTROL_M_DP

If, for technical reasons, it is necessary to use a new technical user, then the name must begin with the string 'Control_M'.

This does not apply to batch input folders created by standard programs.

The convention applies to all productive SAP environments. The implementation for old jobs will take place successively as part of maintenance.

4. Documentation

4.1 General / Disambiguation

There should be the following documentation:

- **Operations Manual**: is the crucial document for system operation
- **Authorization concept:** Definition of user roles and assignment of access authorizations to individual functions or data.
- **User documentation**: User manuals are created by the department or with the support of the department.
  In SAP, it is possible to store application-related documentation. These are objects (ABAP texts, descriptions, HTML documents, ...), which are intended to explain the function (e.g. for data fields), the operation and also the error situation that may occur to the user of a program. If possible and sensible, the possibilities of hypertext should be used to link documents with each other.DDICSAP
- **Technical documentation of the individual objects:** will be created as online documentation for SAP in-house developments in the future (see chapter online documentation of the individual objects).
- **Technical documentation of interfaces / processes:** one document per SAP interface / process. Serves as an overview and summarizes all essential information about an interface / process. In addition, there is the detailed online documentation for each individual object of the interface / process. The documentation can also be done directly.SAP

Attention must be paid to updating the online documentation for each transport order.
In this documentation, the two technical documentations are discussed in more detail. The operating manual, authorization concept and user documentation are not part of this policy.

4.2 Documentation language

The technical documentation (exclusively internal to IT) must be prepared in German or English. In the case of software that is to be used internationally, English should be used as the documentation language.

The language of the original application-related documentation must be determined in consultation with the future users. If the software is used in other countries, a translation must be checked and, if necessary, carried out.

Documentation that can be both technical and application-related (e.g. the description of a data element that can be accessed from the application via the F1 help) must be prepared at least in German or English.

4.3 SAP Online Documentation of Individual Objects

SAP Object	Type of documentation
Domain	Can
Data item	Can/must (if the data element is used on Screens; Reason: Call from F1 help of the application)
Table/Structure/View/Table Type	Kann/Muss
FuBa	Must
BAPIS	Must
Reports	Must
Module pool	Must
News	Kann/Muss
Workflow Object Type	Must
Workflow Tasks	Must
Interfaces / Classes	Must
Method	Must
Transport	Can
Customizing	In this case, the guidelines of chapter 3.2 "Customizing Guidelines" of the Rules of **Procedure and Standards for SAP at FS AG** must be observed. The current version of this document can be found on the intranet > ITAS > Committee.SAP

For the following objects, minimal documentation (e.g. description or short text) is sufficient:

- Transaction
- Screen
- Index
- Search Help
- Development Classes / Packages
- SPA/GPA-Parameter
- Function
- BSP (BusinessServerPages) application, whereby the classes / methods implemented for this purpose must be documented in detail

In the following, hints are given for the structure and content of the documentation for more complex objects.

4.3.1 Function

The following standard structure must be followed in the documentation.

Sub-Items of the Document Structure	Information that must always be included
Short	Automatically transferred to the documentation
Functionality	Does the function module read additional data from the database or does it only process the transfer parameters? What database changes are being made? Where is the commit (in the function module or in the calling program)? What tests are performed?
Examples	
Hints	
Other sources of information	
Parameter	Automatically transferred to the documentation
Exceptions	Automatically transferred to the documentation
Function	Automatically transferred to the documentation

An explanatory long text can be maintained for the function module parameters, which is linked to the function module documentation. The long text can be entered in the parameter tabs (Import, Export, Changing, Tables) by double-clicking in the right-hand column.

4.3.2 Reports / Modulpools

The following standard structure must be followed in the documentation. Which of these is used depends on the complexity of the report.

Sub-Items of the Document Structure	Information that must always be included
Purpose	Brief description of the functionality and data used
Integration (Integration)	Interface data is generated or read, other programs are triggered or called, events are triggered, application logs are written
Prerequisites	
Features (Functionality) selection (Selektion) standard variants (Standard variant) output	Is only data read and output, or is it written to the database? Authorization Sequence logic of the screens
Activities	
Example	

4.3.3 Interfaces / Classes

The following standard structure must be followed in the documentation:

Sub-Items of the Document Structure	Information that must always be included
Short	Automatically transferred to the documentation
Functionality	What (externally relevant) attributes do objects of this class have? What services do objects of this class offer?
Relations	From which class does the described class originate? Which classes are referred to?
Example	
Hints	What should be considered when developing your own subclasses?
Further_Sources_Of_Inf (further information)	

4.3.4 Methods

The following standard structure must be followed in the documentation:

Sub-Items of the Document Structure	Information that must always be included
Short	Automatically transferred to the documentation
Functionality	What is the purpose of the method?
Precondition	What prerequisites / preconditions must be met?
Result	
Parameters	
Exceptions	
Hints	What special features need to be considered when using it?

4.3.5 Transport order (optional)

In the documentation of the transport order, it must be noted that:

- What change / recreation (in the case of changes: description of the main modifications and their effects <what has changed and why>) has been made?

4.3.6 News

Messages (ABAP command **MESSAGE**) are to be created based on the development class / package. Maintaining messages in languages other than the login language is not permitted. The translation is carried out using SAP's translation tools.

Furthermore, long texts in the same language must be created for texts that are to be output as E or A messages. Long texts must also be created for messages where the short text is not sufficient to fully describe the situation/error.

Messages should generally be created as a 'full sentence'. Wildcards may only be used for variables, such as contract number, posting date, etc.

4.4 Development of comprehensive documentation (interface documentation, process documentation)

The documentation is intended to provide a summary overview of an interface / process and all associated components on a few pages. The detailed documentation can then be found with the specified components.

Construction:

- Brief description (type and purpose, systems involved, periodicity, authorizations)
- List of associated components (reports, function modules, tables, views, etc.)
- Application log: name of object and sub-object
- Abort Criteria, Error Handling
- For Interfaces: Description of the Handover Structure

4.5 Notes on SAP Text Maintenance

4.5.1 General texts

Most of the texts that are created for documentation are directly linked to the corresponding development objects.

In addition, texts can also be created independently of development objects, e.g. to store user documentation for a transaction. These texts are processed using transaction **SE61** (ABAP ⇨ Workbench ⇨ Tools ⇨ Tools Documentation) or **SO72**. Your own texts are created under the document classes ***General Text*** (DOKU/TX) or ***Main Chapters of a Structure / Chapter of a Structure*** (BOOK/CHAPTER). The remaining document classes are not to be used for comprehensive documentation in order to avoid overlaps with the documentation of the development objects.

The general texts are displayed by calling the SAP function module **'DSYS_'SHOW** with the parameters dokclass (document class) and dokname (document name).

Tip: If the documentation is to be made available via the Help menu item *for the application* , the TTCDS table can be used for this purpose. Enter the program name, transaction code, documentation class, and document name.

4.5.2 Hyperlinks

Hyperlinks (references) can be included in texts, which make it possible to branch directly into another text module. This option should be used, e.g. to refer from a program description to the documentation of a data element or function module. With this method, a redundant description of objects can be avoided as far as possible.

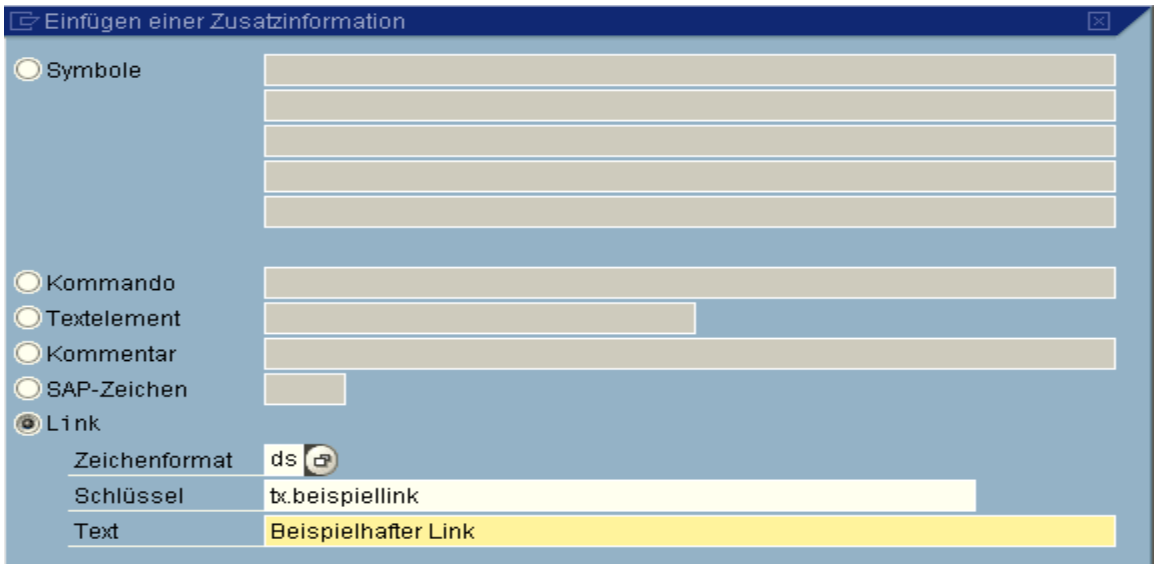
When renaming objects, however, it should be noted that hyperlinks are not automatically updated and there is no proof of use for the affected module.
By inserting references (embedding ⇨ reference), references can be created for many objects. A value support is available here.

For references that cannot be included in this way, a command must be inserted (Edit ⇨ Command ⇨ Insert/Change ⇨ Link). DS must be specified as the character format , and the id of the document class (e.g. TX for text ID, followed by a period and the name of the text to be included (= name of the development object/text) as the key.

Tip: For example, you can find the ID of the document class using the TDCLT table.

For an example of the direct embedding of hyperlinks, see

a) entering the hyperlink



b) das Resultat in dem Dokumentationseditor

4.6 Translation

The documentation for the individual objects can be translated using the central translation tool (SE63). If this is not possible, it is possible to maintain the texts by logging in to the language to be translated. A translation should be carried out at least for the components of the documentation that are available to users. In the case of IT-specific documentation, there is no need for translation.

5. Programming Guidelines

5.1 Readability

This chapter discusses the guidelines for the readability of programs.
In principle, only a maximum of one statement per line may be written.

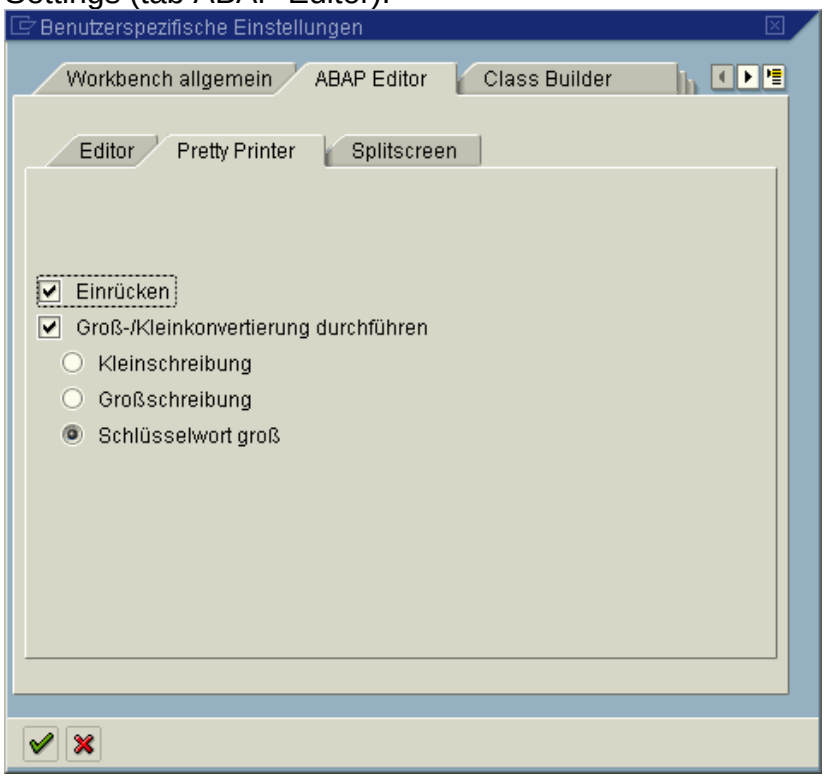
5.1.1 Language

The comment language for inline commenting is German or English.

5.1.2 Pretty Printer

Once new source code has been entered, the "Pretty Printer" must be run to structure the source code. This ensures uniform formatting across all programs. Before activating it for the first time, each program should be formatted by the Pretty Printer.

The settings for the Pretty Printer can be found in the menu of transaction SE80 under Tools → Settings (tab ABAP Editor).



5.1.2.1 Indentation

5.1.2.2 SQL

In order to ensure the readability of database queries via SQL, attention should be paid to orderly readability during construction. In this case, the DB must be submitted as a comment if this is not clear from the name.
After each SQL statement , the SY-SUBRC must be queried, provided that it is capable of providing information about the success or failure of the statement.

Example 1 (SQL statement):

```
  SELECT*
   INTO CORRESPONDING FIELDS OF TABLE lt_return
* -- INIS Kundenstamm
   FROM /vwk/bf2d_0006
   WHERE kivnr_k       IN s_kivnr_k
     AND name1         IN s_name1
     AND name2         IN s_name2
     AND stras         IN s_stras
     AND pstlz         IN s_pstlz
     AND ort           IN s_ort
     AND geb_gruenddat IN s_gg_dat
     AND grossknnr     IN s_grossknnr
      OR telf1         IN s_telf1.

  IF sy-subrc <> 0.
    …
  ENDIF.
```

Example 2 (complex SQL statement):

```
  SELECT DISTINCT *
*  -- Satus
   INTO CORRESPONDING FIELDS OF TABLE lt_status
* -- against status texts
   FROM       /vwk/bf2c_0002t AS c0002t
* -- against team
   INNER JOIN /vwk/bf2c_0001  AS c0001
      ON c0001~product = c0002t~product
     AND c0001~datumbis  >= lv_datum
* -- against SB master record
    INNER JOIN /vwk/bf2c_0009  AS c0009
       ON c0009~product = c0001~product
      AND c0009~sbname    = sy-uname
      AND c0009~datumbis >= lv_datum
      AND c0009~datumvon <= lv_datum
* -- against assignment team -> SB
    INNER JOIN /vwk/bf2c_0008  AS c0008
       ON c0008~product=c0001~product
      AND c0008~team      = c0001~team
      AND c0008~sbname    = c0009~sbname
```

```
      AND c0008~datumbis >= lv_datum
      AND c0008~datumvon <= lv_datum
* -- against REST
    WHERE c0002t~spras = sy-langu
     AND  c0002t~art      = lv_art.

IF sy-subrc <> 0.
  …
ENDIF.
```

5.1.2.3 IF

The following rules apply to IF clauses:
Left and right operands of a plane's comparison operators are left-aligned to start in the same column
the link operator AND or OR of the lowest level is at the end of the statement
Link operators above the lowest level stand alone in individual rows
the join operators above the lowest level are aligned to the left after the opening brackets of the respective layer
Parentheses around expressions associated with AND or OR should be used even if they are not necessary

Example (IF clause):

```
IF ( lv_a = 'x' AND
     lv_b = 'and' )
   OR
   ( ( lv_belart = 'z'  OR
       lv_belnr  = 'x1' )
     AND
     ( lv_belart = 'y1' OR
       lv_belpos = 'y2' ) )
   OR
   ( lv_f = 'y3' AND
     NOT lv_e IS INITIAL ).
```

5.1.2.4 Variable Assignment

If there are multiple variable assignments, the equal sign must always start in the same column.
Multiple assignments are not allowed.

5.1.2.5 Logical Comparison Operators

The following comparison operators are to be used instead of the operators in letter form:

=	for equal
<>	for unequal
>	for larger
<	for small
>=	for greater than or equal to
<=	for less than or equal to

5.1.2.6 Program Header / Method Header / Function Module Head

All programs and implementations of methods should be preceded by a comment block, which should contain the following data:

- Name of the program (for programs only)
- Technical assignment or project
- Brief description of the functionality
- Creator

5.1.3 Program change

In the event of a program change, a comment must be inserted after the program header that contains the following information:

- Fault number, technical order or project
- Brief description of the reason for change
- Modifier

Example of Change Documentation in the Development Object

```
* Wine27. :2001-07-27 Volk's Las, I-seu, Sa-ru 75*
* - new selection option grouping *
```

Further program change notes in coding (commenting out, commenting on the changed lines with transport request number, etc.) can be regulated individually at sub-department and project level. SAP version management offers sufficient technical possibilities to track the changes. The version comparison is available for all objects in the development environment and can also be applied across system boundaries (via RFC connection).

5.1.4 Inline Annotation

Since the objects used should be kept as small as possible, a documentation / comment should also be as small as possible (one to two sentences).
Comments must appear as a block in front of the respective statement. The comment should always refer to the "why", not the "what". The "what" can usually be read from the code.
All comment blocks automatically created by SAP (for example, when creating a) must be filled with comments in a meaningful way.FORM

Own comments in the coding of function modules must always follow the comment block generated by the SAP system with the parameters and (very important!!) must not begin with the two characters *" like the comment block.SAP
Reason: the comment block (starting with *") is completely deleted when the transfer parameters are changed and then reinserted, while the system deletes all lines that begin with the two characters *".SAPSAP

5.2 Structuring

5.2.1 Local/Global Variables

Global variables in TOP includes are to be avoided. They are only allowed if there are no other options, e.g.:

- Session Parameters
- Screen numbers / program names of subscreens
- Control-Variablen: Tabstrip, Tableview (Table Control)

Local variables in PAI/PBO modules are prohibited because they are globally visible. If they are needed, create and call a form routine of the same name.
Form routines must not use global variables. If necessary, they must be passed as parameters in order to document their use.

5.2.2 Constants

Fixed values are to be defined as speaking constants when used several times in coding. (Example: gc_wahr, lc_verarbeitung_erlaubt)

5.2.3 Maximum number of lines

Each modularization unit (form, module, function block, method, ...) must not be longer than 3 screen pages corresponding to 165 lines.

5.2.4 Nesting Depth

IF-Statement: maximum 3
CASE-Konstrukt maximum 2

5.2.5 Section Order / Program Structure

When building programs, the following structure should be used for clarity.

1. Program header
2. Definitions
3. Program Selection Criteria
4. Dates
5. Programm – Forms

The dates in order of their temporal use, if possible:

Example (program structure):

```
initialization.
   perform heat.

*---------------------------------------------------------------------*
 at selection-screen on block blo21.                    "Teamkontrolle
   perform block_21_go using lv_bukrs.
```

```
*---------------------------------------------------------------------*
 at selection-screen.      .           "Buttons und Tabs
   perform buttons_go  using lv_bukrs.

*---------------------------------------------------------------------*
 start-of-selection.
   perform start.
*---------------------------------------------------------------------*
```

This is only to ensure that all points in time are kept together.
Since all variables defined within the points in time are global variables, the coding must be packaged into a form within the points in time. Local variables can then be used here.

5.2.6 Encapsulation

Avoidance of redundancy: The same algorithms or functions must not be implemented in multiple places in an application. They are to be encapsulated in methods, form routines or function modules.

SQL: Insert / Update / Delete

Within a transaction, write SQL statements must be executed in update modules and called up via **CALL FUNCTION func IN UPDATE TASK.** In the context of batch operations with quantity processing, the update module can also be executed synchronously.

The property "Update module" is defined in the properties of a function module:

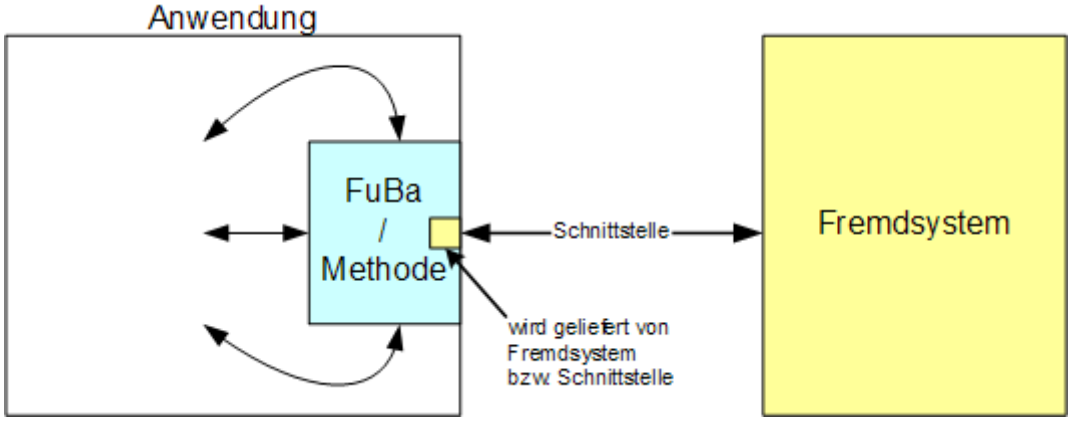
5.2.7 Access to SAP Standard Tables

The existing SAP access modules or interfaces must be used for access. If this option is missing, an in-house development (function module or method) must be carried out in the customer namespace. In case of doubt, the responsible module manager should be contacted.

5.3 Maintainability

5.3.1 Interface characteristics

Access to external systems is encapsulated at one point in the program.



5.3.2 Global Memory

The exchange of data between program parts via the global memory by means of the commands 'Import/Export from memory' is only permitted if the data transfer is not possible by parameter transfers in methods, function modules, etc.

5.3.3 EXEC SQL

The use of database-dependent SQL commands via EXEC SQL is not allowed.

5.3.4 News

The long texts of messages have to be maintained, which makes it easier for users and developers to work in the event of incorrect entries and program errors.

5.3.5 Messages from automatic processes

Batch programs or other processes running in the background must document the processing status by means of messages in the spool or via SAP application logs.

5.3.6 Evaluate SY-SUBRC

If an ABAP statement provides a return code that is informative about the success or failure of the operation, it must be evaluated in the program.

5.3.7 Tables

Among the technical settings for the definition of self-created tables, the size category 0 "Expected data sets: 0 to 22,000" and the data type USER "Customer data type" must be set. Deviations from this are to be agreed with I-SBR.

For revision reasons, all company-owned tables in productive SAP systems must be assigned to an authorization group.
For each table, SAP should define whether it is relevant for accounting. Unclassified tables should be reclassified.
On this basis, system operation is instructed to activate logging for the accounting-relevant tables.
In the future, new tables will be immediately classified and, if necessary, logged.
The classification is carried out by the department involved. The user receives the request for this from the responsible system development unit, which stores this classification in SAP.

5.4 Release Stability

5.4.1 Forward Compatibility

The language elements listed in the chapter "Compilation of Obsolete Language Elements" in ABAP Help under "ABAP - Overview Representations" are no longer permitted.
Obsolete language constructs are no longer allowed, even outside of ABAP Objects, unless a new, valid alternative is specified (see chapter "Replacing Obsolete Language Constructs" in ABAP Help). In particular, this applies to the following instructions:

- TABLES – use explicit table workspaces instead (prohibition of tables with headers)
- OCCURS
- LIKE
- Database accesses without specifying a workspace

Headers to tables are no longer usable, even if they are provided by default.
The TABLES statement and the use of the header is permitted in the context of Dynpro programming (TOP-Include, PBO, PAI) for referencing data structures on the Dynpro.

5.4.2 Backward compatibility

For function modules and interfaces used across applications, a release concept must be used for changes and extensions.
For example, function modules can be provided with release notations (V1_0, V1_1) so that the old version is still available after the introduction of a new release. After a transition period and the decoupled adaptation of the third-party systems, the old version can be deleted.

5.4.3 Customer Modifications

Customer modifications must be approved by the I-control panel before going live. In principle, any manual change of a development object whose name is outside the group-wide namespaces is considered to be a customer modification.

The following exceptions do not require approval by the I-Steering Committee:

- Bug fixes provided by the software vendor (usually SAP) (in advance)
- Explicit enhancements within the SAP Enhancement Framework

Explicit enhancements within the framework of the SAP Enhancement Framework are enhancement options explicitly inserted by the software manufacturer into the source code of individual objects. Implicit extensions, on the other hand, are enhancement options that are available for all non-customer objects of a certain type (for example, for example, for all function modules). (See also "Enhancement Concept" in the SAP documentation for the Enhancement Framework.)

5.5 Scrutinies

5.5.1 Advanced syntax check

The extended syntax check must be performed with all options and must produce an error-free result before the program is released.
Indications must be investigated.

6. Internationalization

6.1 Multi-language capability

6.1.1 General

Text symbols are to be used. When using includes multiple times, texts must only be defined as messages in message classes or must be passed as parameters. Otherwise, the value of the variable 'text-001' depends on the framework program and is therefore indeterminate.
Abbreviations should be avoided because they cause problems, especially during translation.
The translation is carried out using SAP's translation tools.

6.1.2 CIDS-Text

Texts in the DDIC must always be created in the programmer's login language.
For data elements:

- The length of the heading should always be based on the output length of the prepared field (e.g.: date, internally 8 digits (19920728), prepared 10 digits (28.07.1992, 07-28-1992, ...).
- The length of the keywords should be set to 10, 15, 20, regardless of the length of the text in a particular language.

6.1.3 News

Messages (ABAP command **`MESSAGE`**) are handled in the same way as DDIC texts.
Placeholders may only be used for variables such as contract number, posting date, ... be used.
Messages composed of multiple blocks of text are not allowed.

Example:

Message: **&1 &2** is locked.
Call: Message W123(/VWK/020_A)
WITH 'The Contract'(001) DBTAB-VERTRAN.
Edition in German: **`Contract 123 is blocked`**
... in a foreign language: **`??? (Message so untranslatable)`**

All placeholders in a message must be numbered consecutively (necessary for correct positioning in the translation and in the long text).

Example:

Message: The Receipt **&1** cannot be booked.

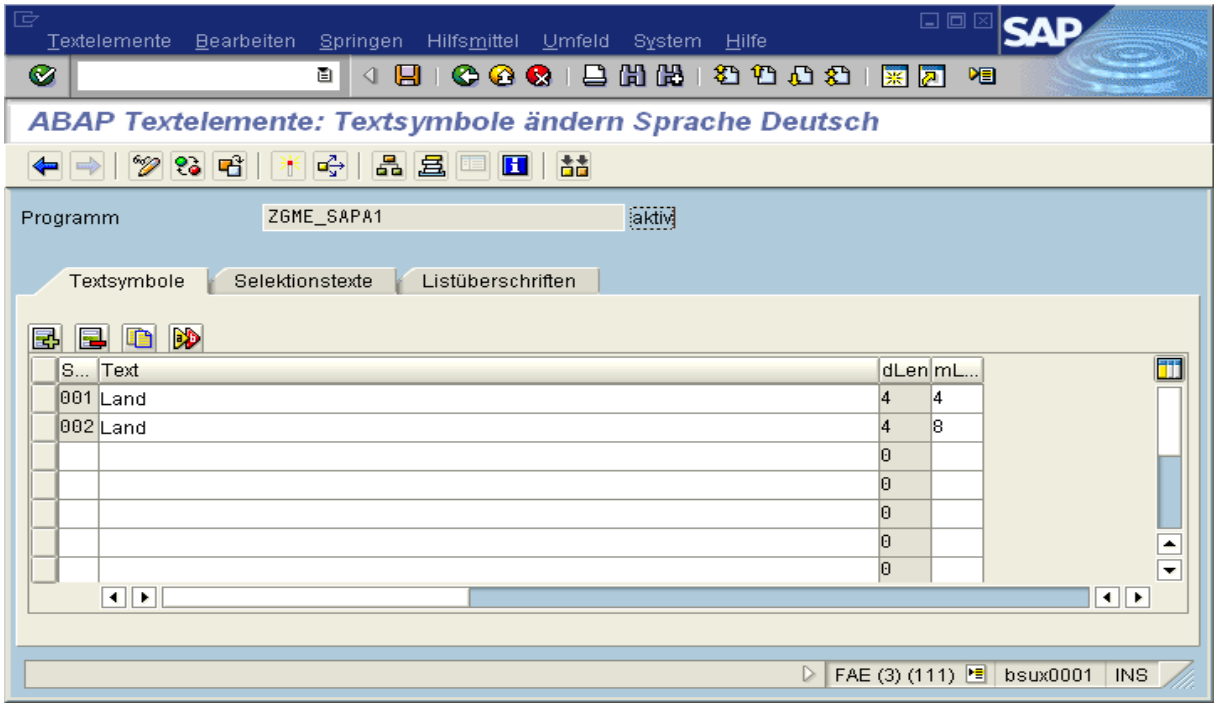
Long: The booking date **&2** in the document **&1** is in the closed posting period **&3**.

6.1.4 Text Elements in ABAP Programs

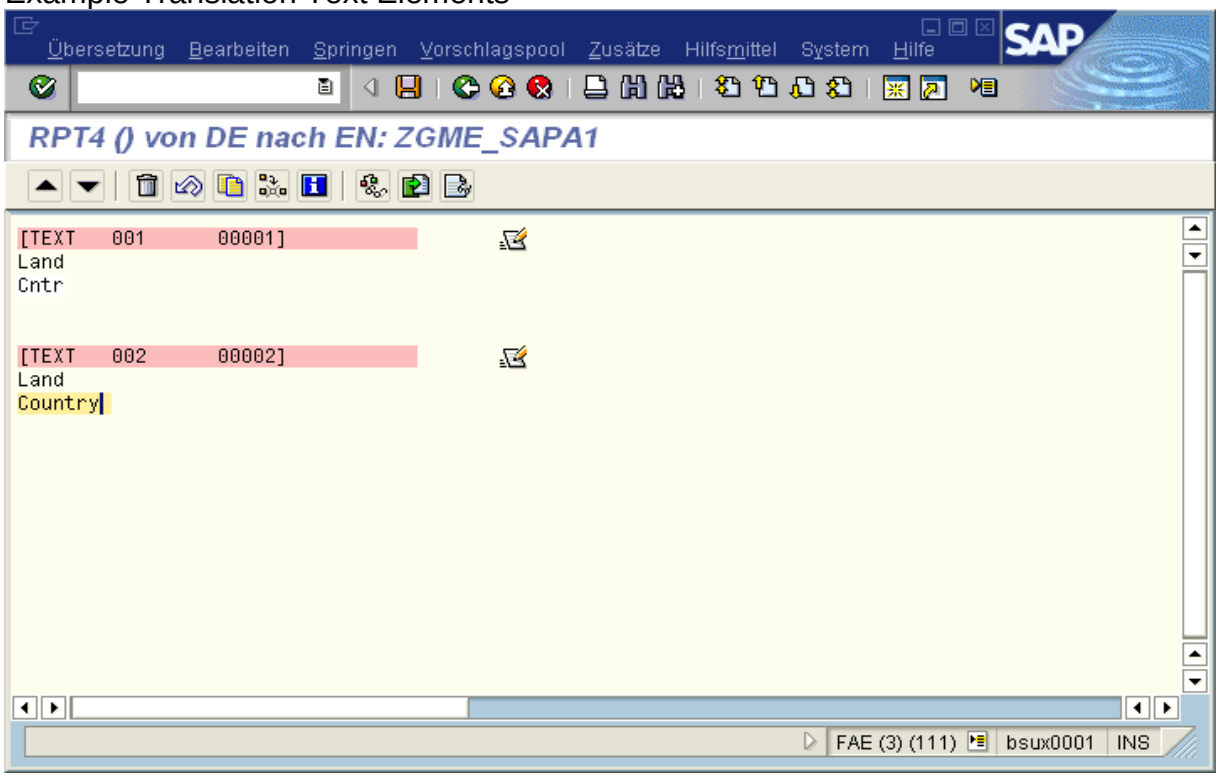
Basically, text elements must be used instead of hard-coded texts. The text elements must be entered in the project language (= login language).

If the user interface allows it, the maximum length of the text elements is significantly greater than the defined length (rule of thumb + 30%).

Example of maintenance of text elements:



Example Translation Text Elements



Text elements may not be linked together in a sentence because this is not comprehensible to translators.

Error example from SAP Style Guide:

Short texts in ABAP:

```
A02 Do you really want the object?
Delete A03?
```

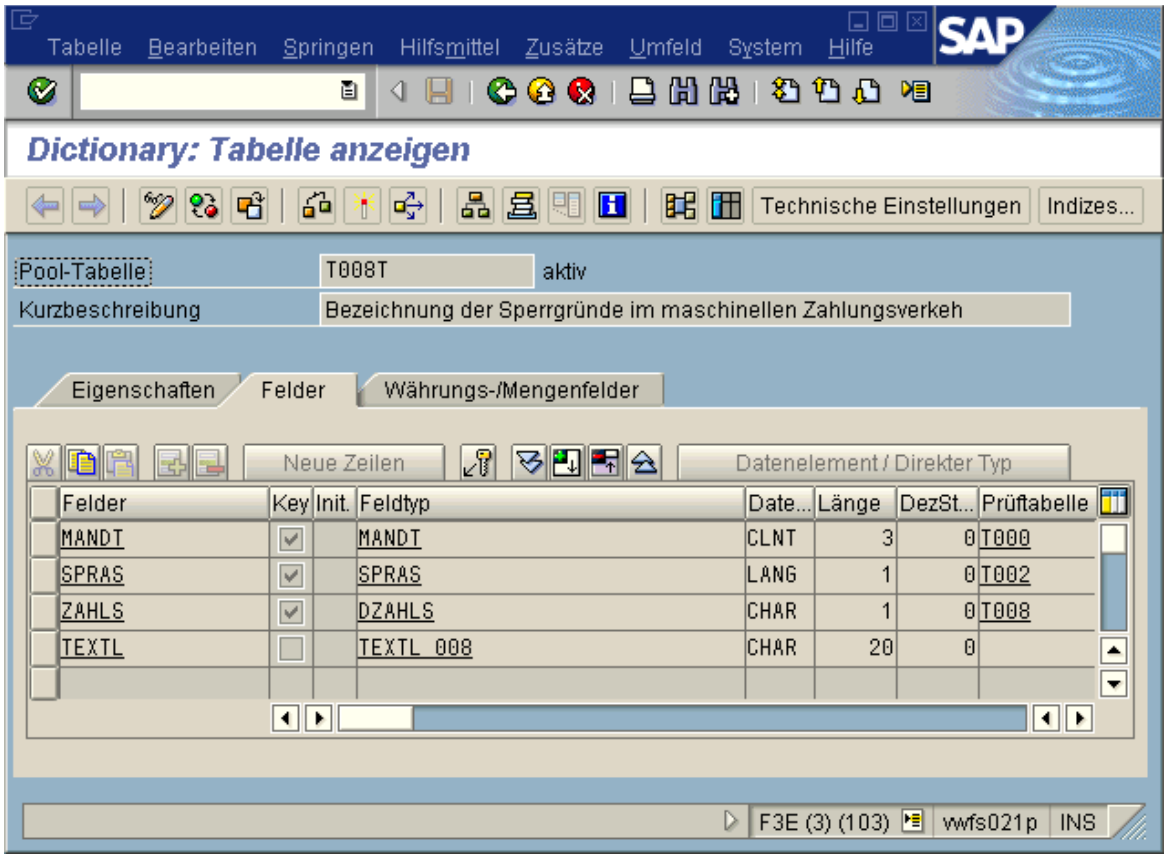
Correct translation in English: Do you really want to delete this object?
Translation of the sub-sentences: Do you really want this object delete?

Longer texts (longer than e.g. 72 characters) can be easily mapped by long message texts, which can BAPI_MESSAGE_GETDETAIL be read by the documented standard function module.

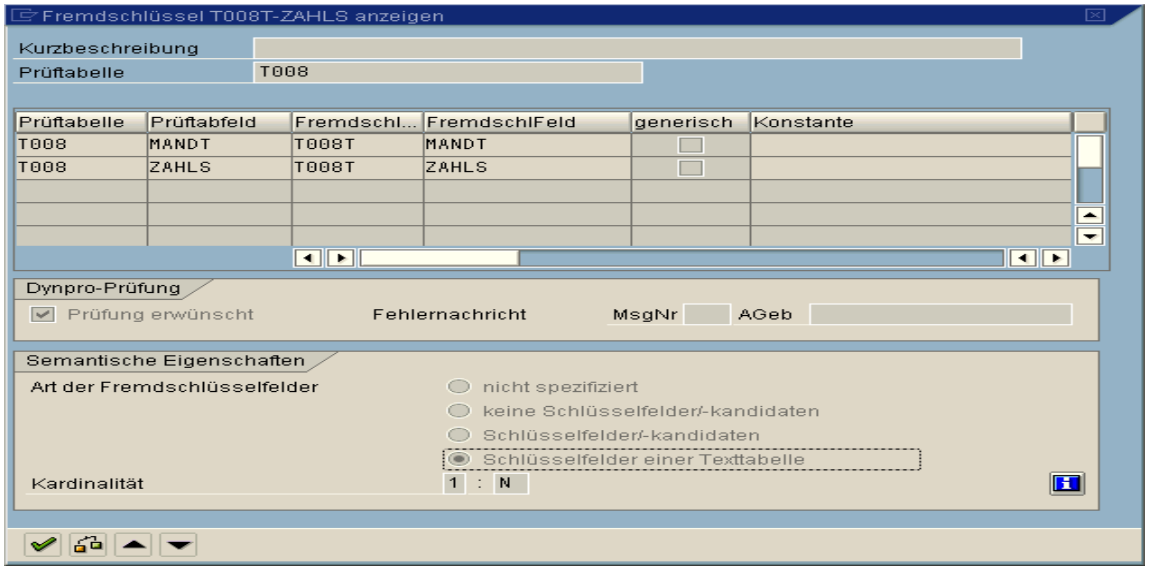
6.1.5 Tables

Texts in Customizing tables must be stored in separate text tables with a language key (SPRAS or LANGU data element).

Example definition of text table:



Example definition of foreign key for text table:

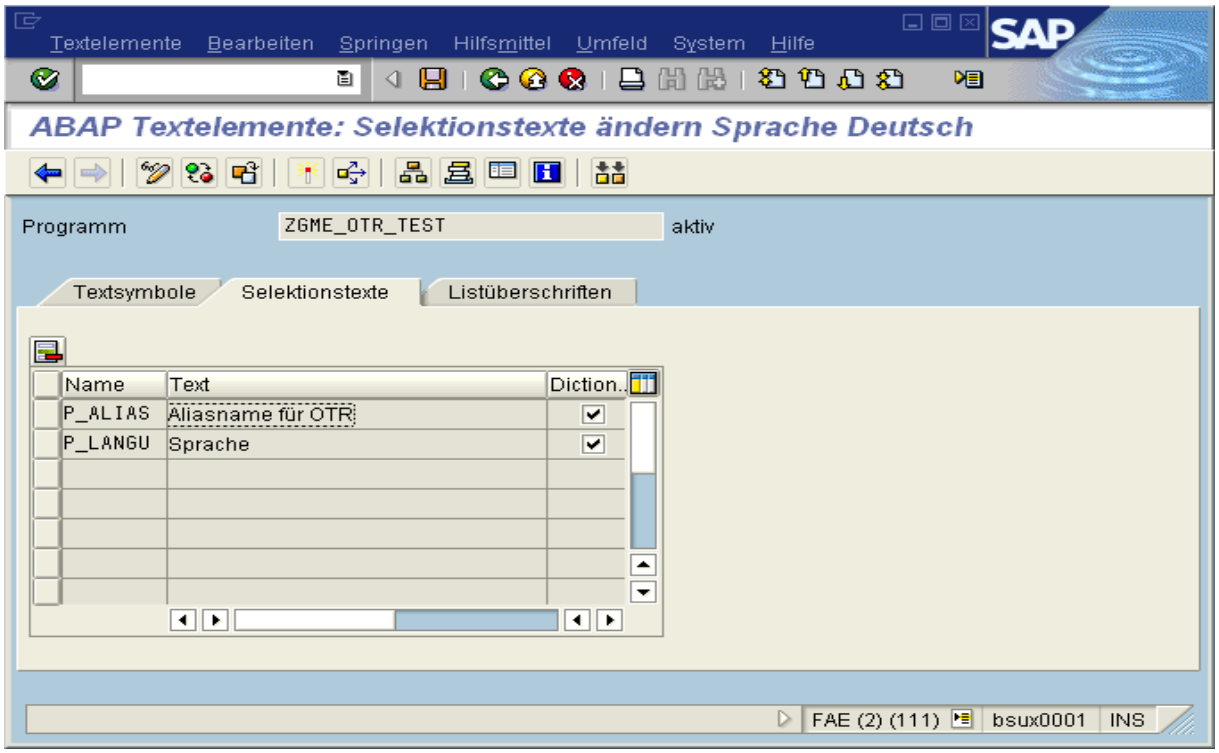


Key fields must be defined so long that different countries can be separated by the start of the key (i.e. character fields must be at least 4 digits long).

6.1.6 Selection Screens

Selection texts are to be copied from the Data Dictionary (if possible).

Example of selection texts:



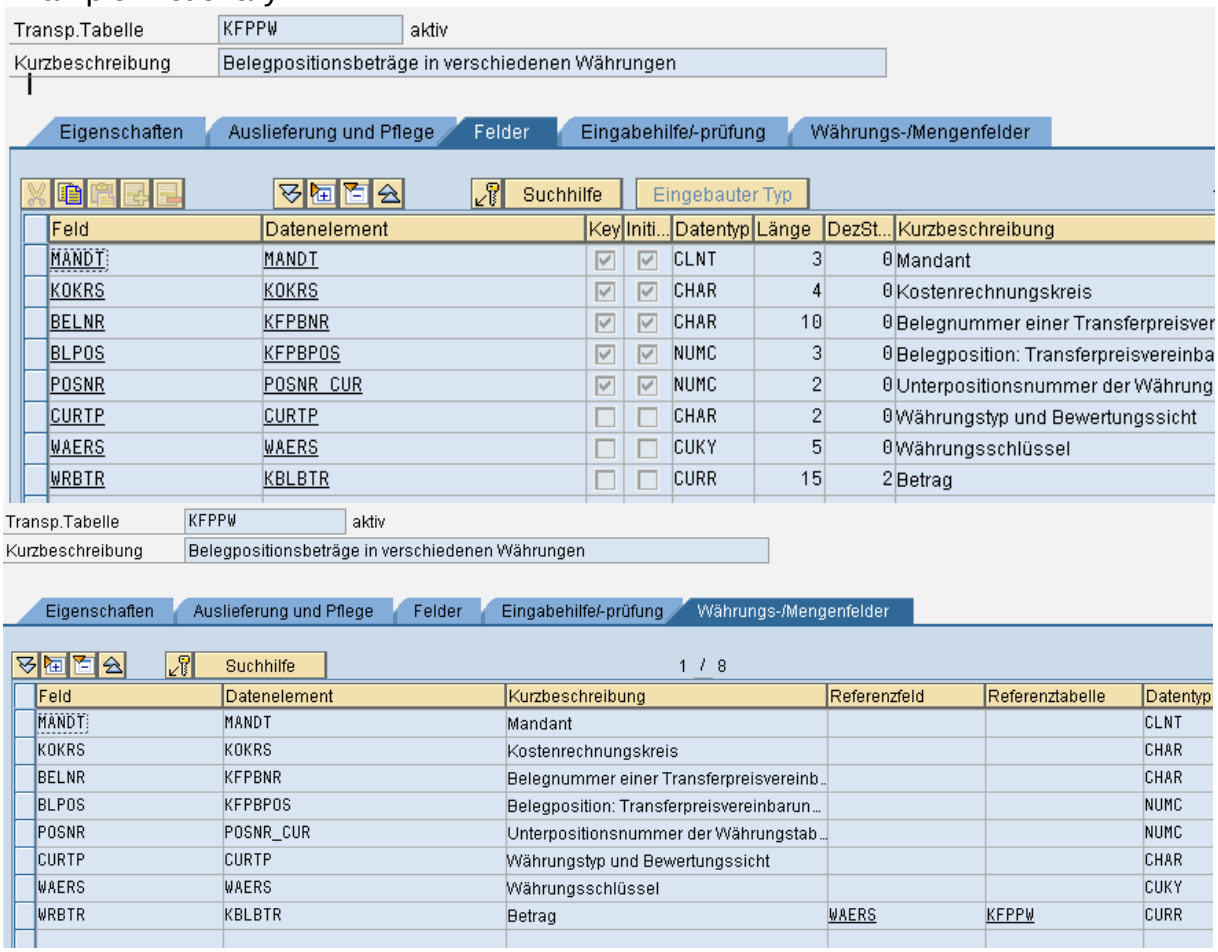
6.2 Multi-currency capability

6.2.1 Currencies in general

- All applications must be designed to work with different currencies.
- Several currencies must be managed for all business transactions that are 'cash-effective' (e.g. contract value, residual values) or lead to documents from external and internal accounting (e.g. FI, CO).
- The underlying SAP application specifies the currencies to be used (e.g. FI, BCA, CML). Binding currencies for in-house developments without reference to a solution are:SAPSAPSAPSAP
  - Transaction currency (currency of the business transaction)
  - Company code currency (currency of the company code, see also table T001)
  - Group currency (currently Euro)

6.2.2 Data Dictionary

For all currency amounts, the corresponding currency must always be entered in structures and database tables. The data type CURR is to be used for currency amounts and the data type CUKY for currencies.
Example Dictionary:



6.2.3 Issuance of currencies

The output of amounts (e.g. by ABAP command WRITE or ABAP List Viewer) takes place when using dictionary objects according to 6.2.2 Data Dictionary always correct (number of decimal places, separators '.' and ','). If no dictionary object is used, the currency field must be named for the currency amount.

Example currency issuance:

```
WRITE: / lv_value CURRENCY lv_curr, lv_curr.
```

6.2.4 Currency conversion

Currency rates are to be requested (for applications that do not run in the FSP) via the "EAI Scenario Currency Rates" (metamodels ExchangeRateRequest and ExchangeRateResponse) (daily updated or real-time).
Currency conversions are carried out using function module
CONVERT_TO_LOCAL_CURRENCY.
If the source and target currencies are not equal to the Group currency (currently euros), the conversion is carried out in the following steps:
Source Currency → Group Currency → Target Currency

6.3 Multi-tenancy

All applications must be designed in such a way that they can be operated in parallel in several SAP clients. The prerequisite for this is compliance with these basic rules:

- The tenant is the first field of each database table in an application (field name: MANDT or CLIENT)
- Database accesses with the addition CLIENT SPECIFIED are not permitted (except for programs for database conversion and client copying)

7. Modularization

7.1 Objective

At this point, the modularization of SAP applications is described from a technical point of view. Technical modularization (use of subroutines, function modules, methods, etc.) is not part of this guideline.
The aim of modularization is a stronger technical decoupling of application parts while maintaining the high level of business integration. This results in greater technical comprehensibility, less maintenance and easier further development of the applications.
Modularization according to development cycles or project planning is prohibited.

7.2 Definition of terms

System: A system runs one or more applications that serve a common business purpose.
Application: Amount of use cases

Use case: IT implementation of a business process, e.g. executing end-of-month posting, creating GP; Synonym to Transaction

Parcel: "A package is a container that contains individual development objects ... and/or other packages."[1]

Object of development: Programs, function modules, tables, screens, data types, etc.

Service: A program or similar that provides services to the outside world via a defined interface. The implementation is the secret of the service.

7.3 Parcels

Packages are always seen as units of business functionalities.

The packages should reflect the business structure of the system[2] , have as few interdependencies as possible and be stable in the long term.

The use of SAP elements should be limited to as few packages as possible.

Only data elements, domains, and function modules from SAP basic packages may[3] be used, unless they are further or additional developments of SAP modules. In this case, elements from these SAP modules may be used. Package usage must be enabled.

7.4 Layered Architecture

If a layered architecture is in place, the separation of interface functionalities (screens, dialog programs, dialog classes) from business functionalities (function modules, subject classes, databases) must be maintained.

Background programs for mass processing must be kept in separate packages or subpackages.

7.5 Modeling

If there is modeling in UML (e.g., objectiF), the packages of the modeling correspond to the packages in SAP. The packet interfaces are derived from the modeled services. If there is a change in the packages in the implementation, this must be followed in the modeling / documentation.

7.6 Design Principles and Rules

Goals

- Determining the offer and protecting against use
- Reduce and clarify dependencies
- Faster and more cost-effective changeability
- Greater independence from development areas inside and outside SAP

Regulate

- "Loose coupling": the lowest possible dependencies between the packets
- Avoidance/elimination of cyclical dependencies
- Parcels must be transportable individually, including their dependent parcels
- Objects with a high frequency of change are to be separated from objects that are stable in the long term by assigning them to different packages
- Heavily referenced packages (multiple use) must be as stable as possible – barely referenced packages can be changed more easily

[1] Prior to release 6.20, only development classes existed, there were no main or subpackages.

[2] To be understood here in the sense of an application.

[3] This refers to packages that are included in SAP's BASIS and ABA packages.

Criteria for defining the packages

1. Stability (long-term)
2. Mapping of the business structure
3. Minimization of dependencies
4. Granularity (not too fine: up to 20 developers per main package)

7.6.1 Procedure for new projects

1. Analysis of the package landscape to be set up:
   1.1. Driven by business management
   1.2. Analysis of interfaces (who uses what?)
2. Definition of packages in the application
   2.1. Defining Main Packages
   2.2. Define Subpackages
3. Offering services via interfaces
4. Documentation of packages and their interfaces

7.6.2 Procedure for the conversion of existing applications

1. Analysis of the package landscape to be set up:
   1.1. Analysis of interfaces (who uses what?)
2. Definition of packages in the application (each existing development class becomes a package)
   2.1. each development class becomes a package
   2.2. Defining Main Packages
   2.3. Define Subpackages
3. Create an "inventory interface" to ensure existing functionality
4. Offering services via "new" interfaces
5. Conversion of the use to the "new" interfaces
6. Documentation of packages and their interfaces

7.7 Literature

- Ackermann, Jörg: "The SAP Package Concept – Experiences in the Modularization of Existing Application Systems" Walldorf (see Intranet -> ITAS -> SAP Committee (News))
- Craig Larman: Applying UML and patterns : an introduction to object-oriented analysis and design and the unified process, 2. ed. - Upper Saddle River, NJ : Prentice-Hall PTR, c2002

8. Security & Privacy

8.1 System Development Requirements

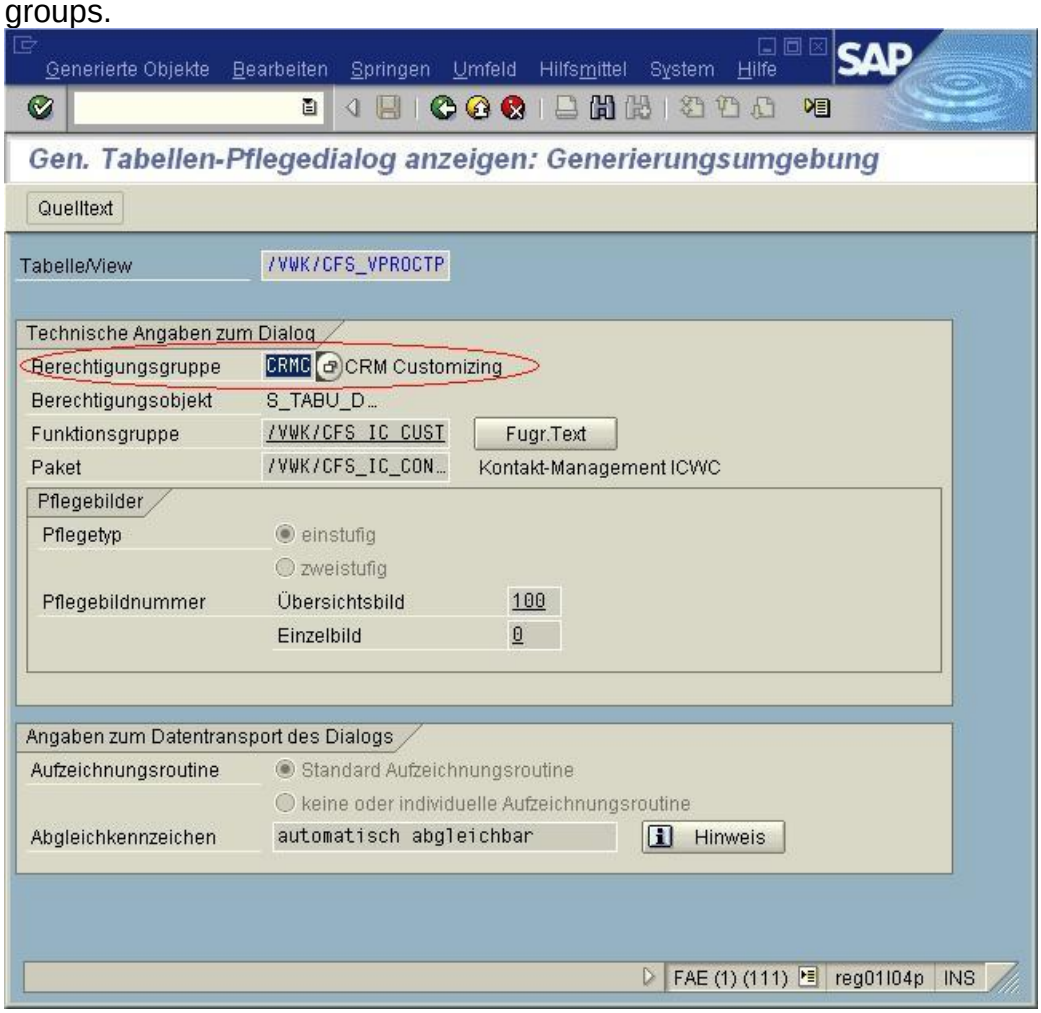
8.1.1 AUTHORITY-CHECK for programs (own and SAP)

In R/3 systems, access protection is essentially based on automatic controls that are stored in the programs. This is the so-called ABAP language element "AUTHORITY-CHECK", which can be stored in the coding of the programs. When a program is executed, this AUTHORITY-CHECK checks whether the authorizations of the calling user are sufficient. In the positive case, access to the information will be allowed; in the negative case, the program must be designed in such a way that access is excluded.

Reliable access protection must be ensured by properly stored AUTHORITY CHECKs in the coding of the programs.

8.1.2 Table Maintenance

All customizing tables developed in-house must be maintained via table maintenance dialogs (i.e., maintenance via SE16 must be prevented!), or maintenance is carried out via independent transactions. Table maintenance dialogs must be provided with context-specific authorization groups.



8.2 Changes to the production system

8.2.1 System Changeability

Transaction **SE06** can be used to control whether objects in the repository and client-independent customizing can be changed. Individual objects, such as:

- Customer Developments
- SAP Basic Components
- Development Workbench

specifically protected. A button assigned to this transaction can be used to evaluate the corresponding change logs. The protection mechanisms defined in this transaction do not affect client-dependent Customizing changes. In this regard, appropriate security mechanisms are defined via client control (see Protection of Table T000).

8.2.2 T000 table protection

Using the corresponding settings in table T000, it is possible to prevent changes to the SAP system in principle or for certain sub-areas. To display or maintain the corresponding settings, transaction **SM30 is used**. There are different settings to be made with regard to the

- Changes for Transports and Client-Dependent Objects
- Changes to Cross-Tenant Objects
- Tenant copy and comparison tool protection

be taken.
In principle, the settings should be chosen in such a way that changes in the productive system are not possible. If it becomes necessary to open up the system with regard to change options, it must be ensured that no uncontrolled changes are made – appropriate documentation should be prepared – and that the changeability status is reset after the settings have been made.

8.2.3 Table Maintenance

The function for manual changes to Customizing tables must be limited to the persons who are responsible for modifying these tables. A general authorization to change all Customizing tables of a system is not permitted.

9. Additional specifications

9.1 General

- Summarize definitions of data objects according to type (TYPES, TABLES, DATA, etc.) for better clarity
- Encapsulate and outsource functionalities - if necessary - not only for reasons of reusability, but also for better readability:
    - If possible, program routines (function blocks; subroutines; methods) no longer than one screen page
    - If possible, no case distinctions (IF, CASE) and loops (LOOP, DO, WHILE) longer than one screen page

- Use blank lines for the better structuring and readability:
  - **Exactly one blank line** - **not none** and **not several blank lines:**
    - To separate case distinctions (IF, CASE) and loops (LOOP, DO, WHILE) from the rest of the instructions
    - Before and after definitions (PROGRAM, FORM, FUNCTION, METHOD)
  - Do not tear apart instructions that belong together with blank lines!
  - Inline documentation:
  - always before processing service blocks; in case of case distinctions within the statement block and not above
  - begin in the column of the corresponding instruction (after Pretty Printer)

Example 1 (Explanations in brackets):

```
"Fallunterscheidung für Funktion (eingerückt vor IF-THEN-ELSE)
IF funktion1.
  "Verarbeitung für Funktion 1 (eingerückt vor dem THEN-Zweig)
  PERFORM subprogram_funktion1 USING gv_variable1.
ELSE.
  "Verarbeitung für Funktion 2 (eingerückt vor dem ELSE-Zweig)
  PERFORM subprogram_funktion2 USING gv_variable2.
ENDIF.

ANWEISUNG.
ANWEISUNG.

"Interne Tabelle … (eingerückt vor LOOP)
LOOP AT …
  ANWEISUNG.
  ANWEISUNG.
ENDLOOP.

ANWEISUNG.
ANWEISUNG.
```

Example 2 (Explanations in brackets):

```
FORM subprogram_funktion1 USING uv_variable type ….

"Fallunterscheidung für Variable(eingerückt vor CASE)
CASE uv_variable.
  WHEN wert1.
    "Verarbeitung für Wert 1(eingerückt vor WHEN)
    ANWEISUNG.
    ANWEISUNG.
  WHEN wert2.
    "Verarbeitung für Wert 2(eingerückt vor WHEN)
    ANWEISUNG.
    ANWEISUNG.
ENDCASE.
ENDFORM.
```

9.2 Use of the ABAP memory

In certain cases, the use of the ABAP memory is required to transfer data between different applications. In order to be able to find the call points for EXPORT and IMPORT again, suitable measures must be taken. When using a structure or a program routine (function module; method) created solely for this purpose, these positions can be found using the proof of use.

10. Programming tips

10.1 General

- With the statement SELECT… PACKAGE SIZE.. WITH key > = gv_key the addition ORDER BY PRIMARY KEY must be used, because otherwise the number of hits is not sorted!
- When processing files on the application server, always work with logical file names:
    - Dynamic Generation via transaction FILE (if necessary, use of the existing logical file names "ZGP_DATEI_IN" and "ZGP_DATEI_OUT")
    - In the program, assemble the file name with the help of the function block “FILE_GET_NAME " (Example: /VWK/ZGP_GPEXT_LADEN)
    - Structure and length of the file name<PRAM_1><CLIENT>0000ZGP<PARAM_2><PARAM_3> _IN.<SYSID> according to convention:
        - <PARAM_1>= ".." (country, 2-digit; mostly "DE")
        - <PARAM_2>= "..." (function-specific; 5-digit part)
        - <PARAM_3>= "..." (data origin; 3-digit; mostly "RVS")
        - Example: /daten/F3D/work/DE1030000ZGPGPEXTINIRVS_IN.F3D
- When validating a file, there are two possible course of action. If immediately before worked with the Function block “FILE_GET_NAME " you can work with the function block "FILE_VALIDATE_NAME". The same parameters can be used there.
  Alternatively, as soon as the path of the file is known, an entry can be made in transaction FILE to store the logical path, the link to the physical path and the definition of a logical file name. The entry for the physical file in the last step remains empty so that the logical file name, which only consists of the physical path, can be specified for the “FILE_VALIDATE_NAME” function block. Thus, only the path is validated and every file name in the path is allowed. Example for a logical path in F3D: "ZDATEN_WORK_VERIFY". The logical file name matches this: "ZALL_VERIFY_FILE".
- With LOOP AT itab to improve performance in general use a field symbol (this saves in case of changes also the MODIFY)
- Runtime analysis (transaction SE30 → Tips & Tricks) for performance analysis
- Always make database changes in dialog applications using the update module
- Long, similar coding can be shortened with generic calls; you can do a lot with field symbols / inheritance!
- Be careful with function groups, the global variables retain the value during LUW, this can cause strange effects in case of function block-calls. Rather use function block only with local variables or even better work with OO classes (the variables are unique for each instance)
- An interface can be created for constants and defined there as SCU _... The constants can then be used system-wide
  (Example: /vwk/if_zgp_konstanten => scu_type_person)

10.2 Update program

- Design the program in such a way that it can also run parallel to the dialog or other changing processes
  -> **Set locks**
- Update programs always with parameters **"Test run"**so that you can test the process without a change function
- Counter independent of the test run functionality
- When processing files in the test run do not create any **OK file**
- From Release 6.40 onwards, programs that carry out a ROLLBACK WORK during a test run, the list is no longer displayed in the job overview (SM37) during background processing, although it is visible via SP01.
  -> Save list lines in the internal table and only output them at the end of the program (example in the program /VWK/ZGP_VERARB_CCF_AV)
- Check when a COMMIT WORK is required:
  - After an internal table has been completely processed (size of the table via constant)
  - Only with complex processing after each rate or operation
- If the fields to be changed are BW-relevant, call function module /VWK/ZGP_FILL_BW_EXTRACT to fill the BW queue (SMQ1)
- Create change documents
  Attention! Do not implement any new change documents, as table CDHDR and CDPOS on the F3P are already “at the limit”!
- Distribution:
  - Check whether the fields to be changed are used in connected systems
  - If so:
    - Check if SKS distribution is required
    - Check if XI distribution is required
    - Integrate distribution modules from function group /VWK/ZGP_GP_AERNDERN as required
    - If an SKS distribution is required, this can be carried out with the function module /VWK/ZGP_BUPA_GPVERT by transferring individual parts via the optional tables parameter IM_PARTS
  - **Attention! With SKS only synchronous distribution!**

10.3 OO Programming

- Classes can inherit. Similar functions can be individually adapted via redefinitions. Example: Cars are inherited by four-wheel vehicles, which in turn are inherited by trucks and cars
- Table controls are replaced by the new ALV technologies
- Only a few public interfaces are defined, processing takes place locally
- Attributes are generally private. Exception: Since external access via getter methods is significantly slower than direct read access to an attribute (approx. 6-7 times slower), simple attributes whose implementation is not expected to change can also be used as "public read-only" or "protected read-only" can be implemented.
- The transfer parameters and method size are handled analogous to the form routines
  → Code returns preferably with returning variables
- References should be defined globally and if possible only be instantiated once
  → Possibly use of static methods.
- Static methods cannot call instance methods and cannot use instance variables.
- The references must be cleared before the transaction is ended (FREE gr _...).

10.4 Initialization of new database fields

There are two possibilities to initialize new database fields:

- Marking the field "initial values" on new field. This action leads to the database conversion during the transport and can therefore only be used for small tables (e.g. customizing tables).
- Writing a program that initializes the field. This is particularly recommended for large tables. The XPRA functionality can be used here by changing the program to which special requirements are placed (no selection screen; loop across all clients with SELECT... CLIENT SPECIFIED...), to be entered in the transport.

If not taking care about them initialization of new database fields, there are often undefined values (NULL) in the field that cannot be used to find the record when selected in the application program.

In case of programs that initialize fields in database tables can lead to errors if a value is queried for <> Space and <>. If the value is NULL, nothing will be found with such a selection.
To avoid this, you should also query "Field is null" when selecting in the initialization program:

Example:
Select DB table where **Field <> Space** and **Field <> X** or **Field is null**.

Of course, you could also read the entire DB table, but the program cannot be restarted (see below). In the event of an error, this is fatal. The same applies if the XPRA is not running because there is an error in the transport. In the event of a restart, field contents that have already been maintained would be deleted again.

There is also a helpful documentation from the creditworthiness colleagues for creating the XPRA:
<<file:\\FSDEBSV00240\projekte\I-SE1\I-SEZ\PP5470-12 Bonitaetssystem\BON4-Systemrealisierung\492_Systemdokumente\XPRA Programme erstellen.doc>>

11. Notes on the system design / DV concept

11.1 Representation of pseudo-code in the system design

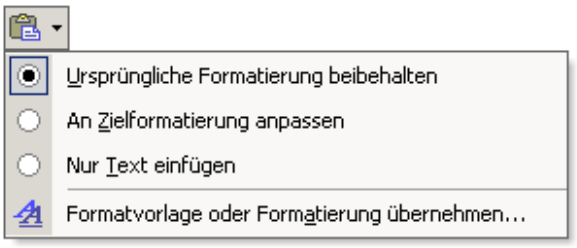
11.1.1 Rules for the representation of pseudo-code

- **General:**
  - ~~Designation of conditions and actions always in plain text (no coding!)~~
  - Use of variable names only if they serve for a better understanding
  - Indentations according to Pretty Printer
- **New creation of a program routine:**
  - Complete presentation of the functionality as pseudo-code in normal script
  - ~~Designation of conditions and actions always in plain text (no coding!)~~
  - Use of variable names only if they serve for a better understanding
- **Change of a program routine:**
  - Representation of a section of the existing coding that is not too long before and after the area to be changed to classify the change
    - *in „Courier New" (Italic)*
      if necessary in font size 9 or 8 to prevent line breaks

- Representation of the area to be changed as
  - **Pseudo-code in "Arial (bold if necessary)"**
  - **`Coding in "Arial monospaced for SAP " (bold)`**
    `if necessary in font size 9 or 8 to prevent line breaks`
- Representation of the coding to be deleted
  - ~~*in „Courier New" (italic, crossed out)*~~

- **Note on copying code from the SAP editor (SE80):**
  When pasting of code in Word the colourful formatting can be avoided, by doing
  - copying into the editor (Notepad) before or
  - the insert With the option "Insert text only" and subsequent formatting:

```
*call°transaction°'/VWK/ZGP_EP_FOR_GP'.¶
¶
```



11.1.2 Example pseudo code - New installation

Action 1

If condition 1 then:
    action 2
    action 3
Otherwise:
    If condition 2 then:
        action 4
    Otherwise:
        Loop over...:
            Action 5
            Action 6
        End of loop
    End if
End if

Action 7

11.1.3 Example pseudo code - modification

Class /VWK/CL_ZGP_SL_UPDATE_RELOC
UPDATE method

```
  . . .
  "Benötigte Felder ermitteln
  ASSIGN COMPONENT 'PARTNER'       OF STRUCTURE i_entry TO <lv_partner>.
  ASSIGN COMPONENT 'ADDRNUMBER'    OF STRUCTURE i_entry TO <lv_addrnumber>.
  ASSIGN COMPONENT 'ADDRESS_GUID'  OF STRUCTURE i_entry TO <lv_address_guid>.

  "Felder für weitere Methodenaufrufe sichern
  ovi_partner        = <lv_partner>.
  ovi_addrnumber_alt = <lv_addrnumber>.* Benötigte Felder ermitteln

  Alte, geprüfte Adresse lesen mit Methode LESEN_ALTADRESSE mit:
                                  <lv_partner>
```

```
                                    <lv_address_guid>
                                    et_bapirettab
                                    Kennzeichen „Adresse Standardadresse“

  Wenn Kennzeichen „Adresse Standardadresse“ = TRUE:
    Aufruf der Methode AENDERN_DATEN
                        …

    "Ändern der Stammdaten
    CALL METHOD me->aendern_stammdaten
      EXPORTING
        iv_pa_nummer  = <lv_pa_nummer>
      CHANGING
        cs_bus000_di  = ls_bus000_di.

    "Ändern der Verteileinträge
    CALL METHOD me->aendern_verteileintraege
      EXPORTING
        iv_partner          = <lv_partner>
        iv_addrnumber_alt   = <lv_addrnumber>
        iv_addrnumber_neu = lv_addrnumber_neu
      IMPORTING
        et_bapirettab       = et_bapirettab
      EXCEPTIONS
        error               = 1.

    IF sy-subrc <> 0.
      EXIT.
    ENDIF.

    "Ändern der Verteileinträge
    CALL METHOD me->aendern_verteileintraege
      EXPORTING
        iv_partner        = iv_partner
        iv_addrnumber_alt = iv_addrnumber
        et_bapirettab     = et_bapirettab
      EXCEPTIONS
        error             = 1.

    IF sy-subrc <> 0.
      EXIT.
    ENDIF.

    "Ändern der Stammdaten
```

11.1.4 Negative example

This is how inserted coding should NOT look like!

```
*&---------------------------------------------------------------------*
*& Report  ZABE_CALL_TX
*&---------------------------------------------------------------------*
*&  Testaufrufe für CALL TRANSACTION
*&---------------------------------------------------------------------*
* 001 2008-03-26 Alfred Behrends, I-SEZ (DKX0NNC)
*     Erstellt
*---------------------------------------------------------------------*
report  zabe_call_tx.

call transaction '/VWK/ZGP_EP_FOR_GP' and skip first screen.

*call transaction '/VWK/ZGP_EP_FOR_GP'.
```